

VHDL-REPOBENCH: A Repository-Level Benchmark for Evaluating Large Language Models on VHDL Design Generation

Prashanth Vijayaraghavan*, Akul Malhotra*, Ashutosh Jadhav, Ehsan Degan, Vandana Mukherjee

IBM Research

San Jose, CA, USA

{prashanthv, Akul.Malhotra}@ibm.com, {ashutosh, edehgha, vandana}@us.ibm.com,

Abstract—Large Language Models (LLMs) are increasingly applied in hardware design automation, demonstrating strong potential in generating and understanding hardware description languages. However, most existing benchmarks focus on Verilog, with limited evaluation of VHDL, which remains widely used in industry and academia for FPGA and safety-critical systems. To address this gap, we introduce VHDL-REPOBENCH, a large-scale, cross-file, repository-level benchmark for assessing LLM capabilities on realistic VHDL design generation and analysis tasks. VHDL-REPOBENCH curates ~ 100 open-source VHDL repositories, encompassing $\sim 2.5k$ VHDL files and ~ 500 testbenches, and provides structured problem statements, module stubs, and self-verifying testbenches. The benchmark enables comprehensive evaluation across syntax, semantic correctness, hierarchical reasoning, cross-file dependency resolution, and functional verification. We evaluate several state-of-the-art models, including GPT-4o, Llama-3-70B, Qwen2.5-72B, CodeLlama-70B, and multi-step reasoning approaches such as Reflexion and CoDes. Results reveal that while current LLMs achieve moderate line- and block-level accuracy, substantial challenges remain in multi-file reasoning, hierarchical design understanding, and specification-to-module generation. VHDL-REPOBENCH represents the first large-scale VHDL-focused benchmark and provides a valuable resource for the hardware design community to evaluate, compare, and advance LLM capabilities for practical VHDL development.

Index Terms—VHDL, RTL, HDL, Repository-Level Benchmark, Large Language Models, Code Generation, Code Summarization, LLM, Functional Verification, Datasets

I. INTRODUCTION

Large language models (LLMs) are rapidly transforming hardware design automation by enabling automated generation, completion, and reasoning over Register-Transfer Level (RTL) code [1]–[7]. While most recent progress has focused on Verilog, VHDL remains widely used across industry, research, aerospace, and safety-critical domains. Compared to Verilog, VHDL enforces a richer type system, stricter modular structure, explicit package usage, and well-defined generic configurations. These features make VHDL more verbose and semantically constrained, demanding deeper structural and cross-file reasoning from LLMs.

Despite the prevalence of VHDL in real-world designs, there is no benchmark evaluating LLMs at the scale and complexity of entire VHDL repositories. Existing VHDL

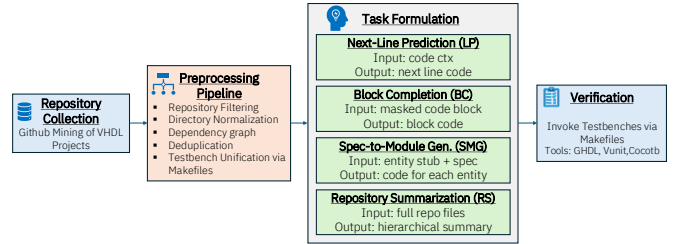


Fig. 1. Overview of VHDL-REPOBENCH. Public GitHub repositories are processed via a preprocessing pipeline to construct the dataset. The benchmark supports 4 tasks: Next-Line Prediction (LP), Block Completion (BC), Specification-to-Module Generation (SMG), & Repository Summarization (RS). Verification module enables evaluation of LLM performance on multi-file VHDL reasoning & generation.

benchmarks, such as VHDL-Eval, focus on small, isolated modules and do not assess model capabilities on project-wide type definitions, shared packages, architectural variations, or hierarchical design. By contrast, recent Verilog benchmarks like RTL-Repo [8] highlight the importance of long-context modeling and cross-file dependency reasoning, but they are Verilog-specific and do not capture VHDL’s unique semantics, compilation constraints, and hierarchical dependencies. In real VHDL repositories, packages define shared types and functions, architectures instantiate entities from other files, generics propagate constraints, and configuration units bind architectures to entities. Without evaluating these aspects, it is impossible to assess an LLM’s readiness for industrial VHDL workflows. Addressing this gap requires a benchmark that preserves full repository granularity rather than reducing VHDL to isolated modules or single files.

In this work, we present VHDL-REPOBENCH, the first large-scale, multi-file, repository-level benchmark specifically for VHDL. We curate ~ 100 open-source VHDL repositories, encompassing $\sim 2.5k$ files and ~ 500 verified testbenches, while maintaining complete project structures including entities, architectures, packages, configuration units, and testbenches. From these repositories, we construct a suite of tasks that reflect realistic RTL design workflows: repository-level line and block completion, specification-to-module generation conditioned on full project context, repository-aware natural-language (NL) summarization, and long-context retrieval for context-driven generation (refer Figure 1). RTL-Repo uses

*These authors contributed equally to this work.

TABLE I
COMPARISON OF VHDL-REPOBENCH WITH PRIOR RTL DATASETS.
DATASET SIZE INDICATES THE NUMBER OF FILES OR PROBLEMS AS
REPORTED. FUNCT. DENOTES FUNCTIONAL VERIFIABILITY AND LANG.
INDICATES THE PROGRAMMING LANGUAGE.

Dataset	Size	Funct.	Cross-File	Lang.
HUMANEVAL [13]	264	✓	✗	Python
REPOCODER [14]	2k	✓	✗	Python
VERILOGEVAL [15]	156	✓	✗	Verilog
VHDL-EVAL [16]	202	✓	✗	VHDL
RTLLM [17]	30	✓	✗	Verilog
RTLREWRITER [18]	95	✗	✓	Verilog
CVDP [19]	783	✓	✓	Verilog
RESBENCH [20]	56	✓	✓	Verilog
TURTLE [21]	1k	✓	✓	Verilog
RTL-REPO [8]	4k	✗	✓	Verilog
VHDL-REPOBENCH	2.5k	✓	✓	VHDL

edit-similarity metrics (Levenshtein distance) without testbench execution, meaning a structurally plausible but functionally broken architecture scores highly. VHDL-REPOBENCH evaluates via GHDL compilation and testbench simulation, measuring functional correctness rather than surface similarity. We evaluate multiple state-of-the-art LLMs, including GPT-4o [9], Llama-3-70B [10], Qwen2.5-72B [11], CodeLlama-70B [12], and domain-tuned variants, and conduct detailed analysis across all tasks. Our experiments reveal that while current models perform reasonably well on localized syntax and line-level completion, they struggle with multi-file reasoning, hierarchical design understanding, and summarization of repository-level design intent.

Contributions: Our key contributions are: (a) the introduction of VHDL-REPOBENCH, the first large-scale, repository-level VHDL benchmark, containing approximately 2.5k real-world VHDL files with preserved hierarchy and semantics; (b) the development of a multi-task evaluation suite covering Next-Line Prediction (LP), Block Completion (BC), Specification-to-Module Generation (SMG), and Repository Summarization (RS) for VHDL; and (c) comprehensive evaluations of state-of-the-art LLMs, revealing core challenges in VHDL reasoning and highlighting research opportunities for next-generation AI-assisted RTL design tools

II. RELATED WORK

Benchmarking large language models (LLMs) for code generation and understanding has progressed rapidly, beginning with software-oriented datasets. Early benchmarks like HUMANEVAL [13] evaluate short, self-contained Python functions, while REPOCODER [14] extends evaluation to the repository-level. Although these datasets have shaped LLM evaluation methodology, they do not reflect hardware design constraints, particularly the concurrency, timing semantics, and cross-file organization inherent to RTL codebases.

Within HDLs, several benchmarks have emerged that focus on functional correctness at the module level. VERILOGEVAL [15] and VHDL-EVAL [16] test single-file RTL generation using simulation-based verification. Related datasets such as RTLLM [17] and RTLREWRITER [18] address syntax

correction and rewriting, but similarly limit evaluation to isolated modules and do not incorporate repository-scale structure. As summarized in Table I, these datasets vary modestly in size and are largely confined to short, verifiable problems without multi-file dependencies. More recent progress examines multi-file reasoning and long-context modeling in Verilog. Benchmarks such as TURTLE [21], CVDP [19], and RESBENCH [20] preserve cross-file module interactions, revealing that state-of-the-art LLMs struggle with dependency resolution, interface consistency, and hierarchical understanding in RTL designs. The largest of these, RTL-REPO [8], aggregates more than 4k Verilog code samples and enables repository-scale code completion and long-context generation. However, all existing repository-level datasets focus exclusively on Verilog, even though VHDL remains central in high-assurance and FPGA-driven development. As Table I illustrates, no prior benchmark combines large scale, cross-file dependencies, and real-world repository structure for VHDL. This absence is particularly limiting because VHDL exhibits design characteristics not present in Verilog, including strong typing, extensive use of packages, explicit generics, and the entity-architecture separation that distributes semantics across multiple files. Existing VHDL datasets are single-file and therefore cannot assess reasoning across packages, architectures, and configuration units, nor can they evaluate retrieval or summarization at the repository level.

To address this gap, VHDL-REPOBENCH introduces the first large-scale, repository-level benchmark dedicated to VHDL. It comprises more than 2.5k real-world VHDL files, preserving complete project hierarchies and cross-file relationships. The benchmark supports tasks such as block and line completion, long-context retrieval, project-level summarization, and specification-to-module generation within authentic repository structure. By placing VHDL on equal footing with previously Verilog-only benchmarks, VHDL-REPOBENCH provides the first opportunity to systematically measure LLM capabilities in type-rich, multi-file, production-grade VHDL environments.

III. VHDL-REPOBENCH

VHDL-REPOBENCH is a repository-level VHDL benchmark for evaluating LLMs on realistic RTL workflows (Fig. 1). It preserves hierarchy, cross-file dependencies, and verification artifacts, enabling tasks such as repository completion, summarization, and specification-to-module generation.

A. Dataset Construction

1) *Repository Collection:* We began by retrieving all public GitHub repositories containing .vhdl or .vhd files, producing an initial pool. To focus on meaningful multi-file designs, we retained only repositories in which at least 70% of RTL files were VHDL, allowing auxiliary files such as testbenches, documentation, or limited Verilog/SystemVerilog sources. This filtering yielded diverse designs across domains including cryptography (AES, PRESENT), processor cores (Neo430, MIPS2), peripherals (LED64, HyperRAM), and accelerators (EdgeDetectionAccelerator). Only permissively licensed projects (MIT, Apache-2.0, BSD) were

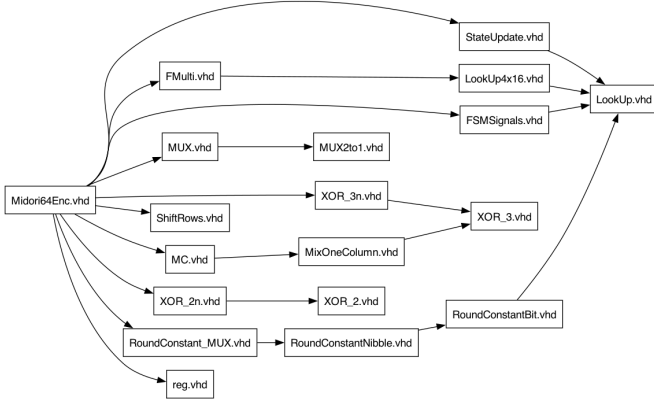


Fig. 2. Sample cross-file dependency graph for *Midori64* block cipher [22], where the top-level file *Midori64Enc.vhd* instantiates submodules such as *FMulti.vhd* and *MC.vhd*.

kept. We removed duplicates via file hashing and pruned forks or near-identical repositories using tree similarity and manual inspection, ensuring each retained project provides unique design content.

2) *Repository Filtering*: To guarantee dataset quality and practical relevance, repositories were filtered according to several criteria. Toy repositories, including those with missing architectures or packages, were removed through a combination of automated heuristics and manual curation. Additionally, repositories were screened for testbenches, which are critical for functional correctness evaluation. Autogenerated code, which often occurs in template-based designs or synthesis tool outputs, was also removed to prevent bias in task evaluation. Following this pipeline, the final dataset comprises ~ 100 repositories with over $\sim 2.5k$ VHDL files, representing a wide spectrum of design complexity and application domains.

3) *Cross-File Dependency Extraction*: Real-world VHDL designs heavily rely on cross-file interactions, such as packages, generics, and configuration units. To systematically capture these relationships, we generate dependency graphs for each repository. Each design is represented as a directed acyclic graph (DAG), where nodes correspond to source files (entities, architectures, or packages) and edges denote instantiation or usage relationships, including: (a) Package imports for shared types, constants, and subprograms; (b) Generic maps connecting entity parameters to architecture implementations; (c) Entity-architecture linking to preserve hierarchical relationships; and (d) Module-level call graphs to track subprogram and procedure usage across files.

These graphs provide a concise view of both hierarchical depth and cross-file breadth, enabling us to filter out trivial or single-file implementations while retaining those with meaningful multi-file structure. Figure 2 illustrates a dependency graph for the *Midori64* block cipher [22], [23], where the top-level file *Midori64Enc.vhd* instantiates submodules present in other files like *FMulti.vhd* and *MC.vhd*, which themselves depend on additional files. This captures the hierarchical organization and cross-file dependencies inherent in complex VHDL designs. By using these dependency graphs as a filtering criterion, we ensure that only repositories exhibit-

ing substantial cross-file interactions are retained, providing realistic challenges for hierarchical reasoning and dependency resolution. These retained graphs also serve as the backbone for task scaffolding, supporting repository-level code completion, summarization, retrieval, and specification-to-module generation tasks.

4) *Dataset Statistics*: Table II summarizes key statistics of VHDL-REPOBENCH. Repositories exhibit substantial cross-file structure: the average cross-file edge count is 29.6 and average maximum dependency depth is 3.4 levels. All $\sim 1.2k$ SMG-Module instances include at least one dependency file confirming that tasks genuinely require cross-file reasoning rather than single-file completion.

B. Verification and Validation

To ensure the correctness and usability of the VHDL-REPOBENCH dataset, we implement a rigorous verification and validation pipeline. For each curated repository, we create Makefiles using a combination of human expertise and LLM assistance. These Makefiles execute the available testbenches and confirm both functional correctness and proper hierarchical connectivity across files. This process is essential because the benchmark tasks: Next-Line Prediction (LP), Block Completion (BC), Specification-to-Module Generation (SMG), and Repository Summarization (RS), depend on accurate multi-file relationships, valid VHDL semantics and verified design functionality. The repositories in the dataset exhibit substantial heterogeneity, including variations in directory layout, file ordering, and testbench type. Some repositories include self-verifying testbenches that automatically check expected behavior for some test cases, while others only compile and run, ensuring proper cross-file dependency resolution. To standardize evaluation across this diversity, the Makefiles encode the correct simulation workflow for each repository, abstracting differences in testbench frameworks such as GHDL, VUnit, or Cocotb [24]. This ensures that every design can be evaluated via a single, uniform make command, with the Makefile internally managing framework-specific and repository-specific compilation and execution steps. By employing this mechanism, we achieve: (a) Functional correctness: each repository can be compiled and executed to verify that the code behaves as intended; (b) Hierarchical and cross-file validation: entity-architecture links, package imports, and submodule instantiations are preserved; (c) Reproducibility: all testbenches can be consistently executed across different

TABLE II
KEY STATISTICS OF THE VHDL-REPOBENCH DATASET. FILES, ENTITIES, ARCHITECTURES, AND TESTBENCHES ARE COUNTED ACROSS ALL ~ 100 REPOSITORIES.

Statistic	Value
# Repositories	~ 100
# VHDL files	$\sim 2.5k$
# Testbenches	~ 500
Avg. designs per repository	1.26
Avg. Cross-file edges per repo	29.6
Avg. Max dependency depth	3.4

Next-Line Prediction (LP)	Block Completion (BC)	Spec.-to-Module Gen. (SMG)	Repo Summarization (RS)
Prompt: You are given a partial VHDL code snippet from the PositAdder module along with other context files from the repository. Generate the next line of code that correctly continues the snippet respecting VHDL syntax,	Prompt: You are given a partially masked VHDL code block from the PositAdder module. Complete the masked block or portion of the code so that the architecture remains syntactically and functionally correct,	Prompt: You are given a high-level design specification and entity stubs . Generate a complete, functionally correct VHDL implementation of the specification, ensuring proper cross-file dependencies...	Prompt: You are given the contents of the PositAdder repository, including entity definitions, architectures, and supporting files. Summarize the core functionality of the repository in natural language,
Input: <pre><context file contents> -- posit16_adder.vhd ... inst_LZcountshift : LZcountshift port map (x => frac_r(14 downto 1), nzeros => nzeros, y => frac_r_shift); -- Add back the sticky bit -- Provide the next line here</pre>	Input: <pre><context file contents> -- posit16_adder.vhd -- Define the scaling factor sf_y <= std_logic_vector(regime_y) & exp_y); -- Check the larger & smaller input process (abs_x, abs_y, sign_x, sign_y, sf_x, sf_y, frac_x, frac_y) -- MASKED BLOCK end process;</pre>	Input: <pre><Specification: Generate the complete VHDL architecture implementing a posit<16,2> adder targeting low-end FPGAs. Entity Stubs: entity posit16_adder is port (x : in std_logic_vector(15 downto 0); y : in std_logic_vector(15 downto 0); r : out std_logic_vector(15 downto 0);</pre>	Input: <pre>Repository File contents: ■ posit16_adder.vhd ■ LZcountshift.vhd ■ right_shifter_extender.vhd ■ posit16_decoder ...</pre>
Output: <pre>frac_r_shift_sticky <= frac_r_shift & frac_r(0);</pre>	Output: <pre>begin if abs_x > abs_y then sign_l <= sign_x; endif</pre>	Output: <pre>Entity implementations of posit16_adder, posit16_decoder, right_shifter_extender, LZcountshift, posit16_encoder</pre>	Output: <pre>The Posit16_Adder repository provides a VHDL implementation of a 16-bit Posit<16,2> adder optimized for low- end FPGAs. The main module, 'posit16_adder', performs addition with proper handling of sign, regime, exponent, fraction alignment, overflow,...</pre>

Fig. 3. Example task formulations for VHDL-REPOBENCH showing the prompt provided to the LLM, input context from the PositAdder repository, and the expected output for each task: Next-Line Prediction (LP), Block Completion (BC), Spec.-to-Module Generation (SMG-R), and Repository Summarization (RS).

repositories using a unified interface; and (d) Dataset quality assurance: invalid, incomplete, or broken repositories are identified and filtered, guaranteeing that downstream tasks (LP, BC, SMG, RS) are evaluated on reliable data. This verification step distinguishes VHDL-REPOBENCH from larger datasets like RTL-Repo, which rely on edit similarity and exact match without testbench execution, potentially reducing quality. By enforcing functional execution and cross-file correctness, our dataset provides a more reliable benchmark for VHDL code understanding and generation.

C. Task Formulation

To evaluate LLM capabilities on multi-file VHDL repositories, we define four tasks that capture both localized code generation and repository-level reasoning. Each targets key aspects of VHDL understanding, including syntax, semantics, hierarchy, and design intent. The tasks are as follows:

Next-Line Prediction (LP): Predict the next line of VHDL code given preceding context within a file or across files.

Block Completion (BC): Generate larger coherent code regions, including masked code blocks, entire processes, or complete files.

Repository Summarization (RS): Produce structured NL summaries of a repository’s functionality, hierarchy, and interconnections across its entities, architectures, and packages.

Specification-to-Module Generation (SMG): Generate functional VHDL from specifications and repository context. *SMG-Module* (SMG-M) generates entity architectures, while *SMG-Repo* (SMG-R) generates complete RTL repositories from system specifications. Dependency graphs preserve cross-file hierarchy and interactions. All datasets are curated from VHDL repositories and quality-verified.

1) *Next-Line Prediction (LP):* The Next-Line Prediction task evaluates the model’s ability to generate the subsequent line of VHDL code given preceding lines. It measures immediate syntactic correctness, type consistency, and local semantic validity in both local-file and cross-file contexts.

Input: VHDL snippet containing all preceding lines within a process, architecture, or entity. Context can include relevant lines from other files in the repository (cross-file).

Output: The predicted next line that correctly continues the snippet, respecting syntax, types, and dependencies.

Data Preparation: Examples are sampled from architecture and process blocks across curated repositories. To focus on meaningful lines, we exclude structural keywords (e.g., begin, end), comments, and simple signal declarations, selecting lines that contribute substantive computation or control flow. Autogenerated or trivial lines are further filtered using file-hash deduplication. A subsample of the dataset is qualitatively verified to ensure high quality.

Metrics: Exact match accuracy and Pass@k based on verification success (see Sec. III-B).

2) *Block Completion (BC):* Block Completion evaluates hierarchical and cross-file reasoning by generating contiguous regions of code. This task assesses both local coherence and global repository-level consistency, requiring models to respect inter-file dependencies and architectural hierarchy.

Input: An architecture, entity, or file with masked code blocks.

Output: Completed code blocks that correctly integrate with both local context and cross-file dependencies, including packages, generics, and instantiations of other entities.

Data Preparation: Candidate blocks are selected using repository dependency graphs, prioritizing regions whose removal breaks verification or synthesis. Masked blocks are validated through syntax checks and self-verifying testbenches with dependent files. Cross-file context includes relevant packages, entity interfaces, and generics from repository relationships rather than arbitrary file concatenation.

Metrics: BC is evaluated using Pass@1 and Pass@3 based on GHDL compilation success.

3) *Repository Summarization (RS):* Repository Summarization (RS) evaluates a model’s ability to generate structured, high-level descriptions of a VHDL repository by capturing its functionality and hierarchical organization. Unlike code-generation tasks, RS focuses on design-intent understanding

TABLE III
SUMMARY OF TASKS IN VHDL-REPOBENCH, INCLUDING APPROXIMATE SIZE, INPUT, AND OUTPUT FORMAT. HERE, SMG REFERS TO SPECIFICATION-TO-MODULE GENERATION (SMG)

Task	Size (approx.)	Input	Output
Next-Line Prediction (LP)	~ 7k	Preceding lines (local/cross-file context)	Next line of code
Block Completion (BC)	~ 1k	Partial file or masked block	Completed block/file
Repository Summarization (RS)	850	Repository files	NL summary
SMG-Module (SMG-M)	~1.2k	Entity stub + Specification + Cross-file Context	Architecture Body
SMG-Repo (SMG-R)	20	System-Level Spec + Entity Signatures	Complete RTL source

and is therefore excluded from Pass@k compilation evaluation. Its primary purpose is to assess whether models can reason about and explain repository-level structure, a foundational capability for specification-driven hardware design.

Input: Complete repository files including entities, and packages, preserving cross-file dependencies.

Output: A coherent NL summary that explains the functionality, design intent of modules/files within the repository.

Data Preparation: Summaries and specifications are initially generated using LLMs (e.g., DeepSeekV3 [25]) from repository code, comments, and documentation. To ensure quality and fidelity, about 10% of repositories are manually audited, confirming that the summaries accurately capture hierarchical structure, module interactions, and design intent.

Metrics: Summarization quality is evaluated using ROUGE-L for lexical overlap and structural alignment, along with a *Preferred Rate (PR)*, where an LLM acts as a judge to assess correctness of the summary relative to the repository context.

4) *Specification-to-Module Generation (SMG)*: SMG evaluates a model’s ability to generate functional VHDL architectures from high-level design specifications, correctly resolving cross-file dependencies and integrating with existing modules and packages. We provide two variants spanning entity-level and system-level generation.

SMG-Module targets one VHDL entity per instance. The model receives an entity stub (ports, generics), a natural-language specification, and cross-file context assembled from the repository dependency graph, and must generate the complete architecture body. This is the primary evaluation target for statistical comparisons.

SMG-Repo targets end-to-end system design. The model receives a system-level functional specification, top-level entity signature, and package stubs, and must generate *complete* RTL source files.

Data Preparation: NL specifications are generated by DeepSeekV3 [25] from repository code and documentation; ~10% of instances are manually audited for fidelity. All instances are verified by GHDL compilation. The 135 SMG-Module instances with testbenches are additionally validated by testbench execution, confirming functional correctness and cross-file dependency preservation. Grounding specifications in real repository structure mitigates LLM self-alignment bias noted in prior work.

Metrics: Pass@1 via the unbiased estimator [13], based on GHDL compilation success; testbench execution is additionally required for the verified subset.

IV. EXPERIMENTS

We evaluate several LLMs on VHDL-REPOBENCH to assess VHDL code generation and summarization. The evaluation suite consists of the four tasks in Section III-C: LP, BC, SMG, and RS. Evaluated models include GPT-4o, Verigen, CodeLlama-70B, and Qwen2.5-72B. We also study the effect of structured multi-step reasoning using *Reflexion* [27] and *CoDes* [26] applied to Qwen2.5-72B.

Data Preparation. LP and BC examples are sampled from curated repositories using dependency graphs to ensure that selected lines or blocks are semantically meaningful and would break compilation or testbench execution if removed. SMG pairs entity stubs with generated specifications, and correctness is validated through compilation and execution of self-verifying testbenches. RS prompts are produced using LLMs (CodeLlama-70B [12] and GPT-4o [9]) and manually checked on approximately ten percent of repositories to ensure high-quality summaries. For all tasks, models are prompted in a standardized setting. Code-generation tasks use sampling with temperature 0.2, top- k of 50, and top- p of 1.0. Context limits follow model constraints: approximately 128K tokens for GPT-4o, Llama-3-70B, and Qwen2.5-72B; 16K for CodeLlama-70B; and 2K for Verigen. Limited context windows disadvantage certain models on large repositories, reflecting practical deployment constraints. To benchmark multi-step reasoning, we employ *Reflexion* [27] and *CoDes* [26], which generate intermediate reasoning steps. This enables systematic evaluation of complex VHDL tasks and a model’s ability to decompose problems. Code generation tasks use standardized sampling (temperature 0.2, top- k = 50, top- p = 1.0) and context limits of ~ 128K tokens (GPT-4o, Llama-3-70B, Qwen2.5-72B), 16K (CodeLlama-70B), and 2K (Verigen). Limited context windows may disadvantage models on large repositories. Since data is sourced from public GitHub repositories, training contamination is possible. Despite deduplication, overlap with pretraining corpora cannot be fully ruled out, affecting LP and BC interpretation.

Evaluation Metrics. LP is evaluated using Exact Match and Pass@1 (compilation success) [13]. For SMG and BC, Pass@1 requires GHDL compilation, and for a verified subset, passing testbench execution. RS is evaluated using ROUGE-L and Preferred Rate (PR). Unlike RTL-Repo’s Levenshtein edit distance, which measures surface similarity, our Pass@k metrics assess compilability and functionally correctness.

TABLE IV
PERFORMANCE OF EVALUATED LLMs ON VHDL-REPOBENCH. LP: NEXT-LINE PREDICTION, BC: BLOCK COMPLETION, SMG: SPECIFICATION-TO-MODULE GENERATION, RS: REPOSITORY SUMMARIZATION. CONTEXT INDICATES MAXIMUM TOKENS THE MODEL CAN PROCESS. CoDES [26] AND REFLEXION [27] SHOW COMPETITIVE IMPROVEMENTS OVER BASE QWEN2.5-72B.

Model	Context (tokens)	LP		BC (Pass@1)	SMG (Pass@1)		RS	
		Exact Match	Pass@1		SMG-R	SMG-M	ROUGE-L	PR
GPT-4o	128K	59.2	60.9	45.1	19.8	41.3	44.5	0.49
Qwen2.5-72B	128K	48.7	53.1	38.2	15.0	34.8	37.6	0.40
Llama-3-70B	128K	48.3	48.9	36.9	14.2	33.1	38.4	0.41
CodeLlama-70B	16K	39.2	40.3	29.8	10.2	21.4	28.7	0.26
Verigen	2K	26.4	24.8	17.5	6.5	12.8	-	-
Qwen2.5-72B + Reflexion	128K	56.2	56.7	41.7	16.5	37.2	39.5	0.43
Qwen2.5-72B + CoDes	128K	55.4	57.2	40.4	16.8	36.9	38.6	0.41

V. RESULTS

Table IV shows performance across all four VHDL-REPOBENCH tasks. Three trends emerge: (1) large-context models achieve higher repository-level accuracy, (2) GPT-4o is the strongest overall, and (3) multi-step prompting improves Qwen2.5-72B across tasks.

Impact of Context Length

Models with 128K-token context windows (GPT-4o, Qwen2.5-72B, Llama-3-70B) consistently outperform smaller-context models. Long-context access is essential for VHDL, where correctness frequently depends on type declarations, package imports, and hierarchical instantiations distributed across multiple files. In contrast, CodeLlama-70B (16K) and Verigen (2K) exhibit sharp performance degradation, particularly in BC and SMG, where missing cross-file information directly limits correctness. This confirms that repository-scale HDL tasks fundamentally require long-context modeling.

Impact of Structured Multi-Step Reasoning

Applying Reflexion [27] and CoDes [26] to Qwen2.5-72B yields meaningful improvements across LP, BC, and SMG. Reflexion helps reduce syntactic inconsistencies through iterative self-critique, while CoDes decomposes repository-level structure into intermediate descriptions before generation. Although neither surpasses GPT-4o, both significantly narrow the gap relative to the base Qwen2.5-72B model, demonstrating the value of structured reasoning.

Task-Wise Analysis

Next-Line Prediction (LP). LP evaluates fine-grained code understanding. GPT-4o achieves the best performance (60.9 Pass@1), with Qwen2.5-72B and Llama-3-70B following closely. CodeLlama-70B & Verigen struggle due to truncated context and weaker VHDL-specific priors. Reflexion and CoDes boost Qwen2.5-72B by improving line-level consistency and reducing hallucinated identifiers.

Block Completion (BC). BC requires generating syntactically and semantically coherent multi-line segments. GPT-4o leads with 45.1 Pass@1, benefiting from long-range dependency tracking. Context-limited models degrade substantially, confirming the necessity of cross-file visibility. Reflexion provides the largest improvement for Qwen2.5-72B on this task by enabling iterative structural correction.

Specification-to-Module Generation (SMG). SMG is the most challenging task. For SMG-Repo, which requires generating all RTL files from a system-level specification, GPT-4o achieves Pass@1 of 19.8, with long-context models substantially outperforming short-context ones. For SMG-Module (entity-level generation across $\sim 1.2k$ instances), GPT-4o reaches 41.3, confirming that entity-level generation is meaningfully more tractable than full-repository synthesis. Reflexion and CoDes improve Qwen2.5-72B on both variants by decomposing repository structure before generation.

Repository Summarization (RS). RS evaluates a model’s ability to explain design hierarchy and module intent. GPT-4o achieves the highest ROUGE-L and PR. Qwen2.5-72B and Llama-3-70B perform well with full context, while CodeLlama-70B underperforms due to a limited window. Reflexion and CoDes give modest but consistent gains by promoting structured decomposition.

Error Analysis. Three recurring failures appear: (i) incorrect cross-file references (missing imports/entity mismatches), (ii) structurally correct but functionally failing modules, and (iii) degraded performance on long dependency chains, highlighting limits in hierarchical reasoning and global consistency.

VI. CONCLUSION

We introduce VHDL-REPOBENCH, the first benchmark for evaluating LLMs on repository-scale VHDL understanding, generation, and cross-file reasoning. Tasks mirror realistic workflows: next-line prediction, block-level completion, specification-driven module or repository generation, and repository summarization, capturing challenges absent in file-level HDL datasets. Long-context models (GPT-4o, Qwen2.5-72B, Llama-3-70B) outperform shorter-context models, showing the value of repository context for hierarchical and cross-file reasoning. GPT-4o achieves the highest accuracy, while multi-step prompting strategies like Reflexion and CoDes boost Qwen2.5-72B. Yet, performance on specification-to-module generation and block-level reconstruction remains insufficient for reliable RTL automation. VHDL-REPOBENCH lays a foundation for evaluating EDA-focused LLMs and highlights directions for future research: better structural reasoning, broader context integration, and hardware-aware constraints.

REFERENCES

- [1] Z. Yu, M. Liu, M. Zimmer, Y. Celine, Y. Liu, and H. Ren, "Spec2rtl-agent: Automated hardware code generation from complex specifications using llm agent systems," in *2025 IEEE International Conference on LLM-Aided Design (ICLAD)*, pp. 37–43, IEEE, 2025.
- [2] H. Pearce, B. Tan, and R. Karri, "DAVE: Deriving Automatically Verilog from English," in *Proceedings of the 2020 ACM/IEEE Workshop on Machine Learning for CAD (MLCAD)*, p. 27–32, Association for Computing Machinery, 2020.
- [3] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-Chat: Challenges and Opportunities in Conversational Hardware Design," in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*, pp. 1–6, IEEE, 2023.
- [4] Y. Zhang, Z. Yu, Y. Fu, C. Wan, and Y. C. Lin, "MG-Verilog: Multi-grained Dataset Towards Enhanced LLM-assisted Verilog Generation," in *The First IEEE International Workshop on LLM-Aided Design (LAD)*, 2024.
- [5] M. Akyash, K. Azar, and H. Kamali, "Rtl++: Graph-enhanced llm for rtl code generation," *arXiv preprint arXiv:2505.13479*, 2025.
- [6] Y. Zhao, D. Huang, C. Li, P. Jin, Z. Nan, T. Ma, L. Qi, Y. Pan, Z. Zhang, R. Zhang, *et al.*, "Codev: Empowering llms for verilog generation through multi-level summarization," *arXiv preprint arXiv:2407.10424*, 2024.
- [7] J. Zuo, J. Liu, X. Wang, Y. Liu, N. Goli, T. Xu, H. Zhang, U. R. Tida, Z. Jia, and M. Zhao, "Complexvcoder: An llm-driven framework for systematic generation of complex verilog code," *arXiv preprint arXiv:2504.20653*, 2025.
- [8] A. Allam and M. Shalan, "Rtl-repo: A benchmark for evaluating llms on large-scale rtl design projects," in *2024 IEEE LLM Aided Design Workshop (LAD)*, pp. 1–5, IEEE, 2024.
- [9] OpenAI, "Chatgpt," <https://chatgpt.com/>, 2024.
- [10] AI@Meta, "Llama 3 Model Card," 2024.
- [11] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, *et al.*, "Qwen technical report," *arXiv preprint arXiv:2309.16609*, 2023.
- [12] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [13] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [14] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "Repocoder: Repository-level code completion through iterative retrieval and generation," *arXiv preprint arXiv:2303.12570*, 2023.
- [15] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "VerilogEval: Evaluating Large Language Models for Verilog Code Generation," in *2023 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, IEEE, 2023.
- [16] P. Vijayaraghavan, L. Shi, S. Ambrogio, C. Mackin, A. Nitsure, D. Beymer, and E. Degan, "Vhdl-eval: A framework for evaluating large language models in vhdl code generation," *arXiv preprint arXiv:2406.04379*, 2024.
- [17] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTLML: An Open-Source Benchmark for Design RTL Generation with Large Language Model," in *Proceedings of the 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pp. 722–727, IEEE, 2024.
- [18] X. Yao, Y. Wang, X. Li, Y. Lian, R. Chen, L. Chen, M. Yuan, H. Xu, and B. Yu, "RTLRewriter: Methodologies for Large Models aided RTL Code Optimization," in *2024 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, IEEE, 2024.
- [19] N. Pinckney, C. Deng, C.-T. Ho, Y.-D. Tsai, M. Liu, W. Zhou, B. Khailany, and H. Ren, "Comprehensive verilog design problems: A next-generation benchmark dataset for evaluating large language models and agents on rtl design and verification," *arXiv preprint arXiv:2506.14074*, 2025.
- [20] C. Guo and T. Zhao, "Resbench: A resource-aware benchmark for llm-generated fpga designs," in *Proceedings of the 15th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies*, pp. 25–34, 2025.
- [21] D. Garcia-Gasulla, G. Kestor, E. Parisi, M. Albert'i-Binimelis, C. Gutierrez, R. M. Ghorab, O. Montenegro, B. Homs, and M. Moretó, "Turtle: A unified evaluation of llms for rtl generation," *CoRR*, 2025.
- [22] S. Banik, A. Bogdanov, T. Isobe, K. Shibutani, H. Hiwatari, T. Akishita, and F. Regazzoni, "Midori: A block cipher for low energy," in *International Conference on the Theory and Application of Cryptology and Information Security*, pp. 411–436, Springer, 2015.
- [23] A. Aghaie, A. Moradi, S. Rasoolzadeh, A. R. Shahmirzadi, F. Schellenberg, and T. Schneider, "Impeccable circuits," *IEEE Transactions on Computers*, vol. 69, no. 3, pp. 361–376, 2020.
- [24] B. J. Rosser, "Cocotb: a python-based digital logic verification framework," in *Micro-electronics Section seminar. CERN, Geneva, Switzerland*, 2018.
- [25] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.
- [26] P. Vijayaraghavan, A. Nitsure, C. Mackin, L. Shi, S. Ambrogio, A. Haran, V. Paruthi, A. Elzein, D. Coops, D. Beymer, *et al.*, "Chain-of-descriptions: Improving code llms for vhdl code generation and summarization," in *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*, pp. 1–10, 2024.
- [27] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Proceedings of the 37th International Conference on Neural Information Processing Systems (NeurIPS)*, Curran Associates Inc., 2024.